

C Programming and Data Structure Lab Manual

Index

S. No	Program	Page No.
1	Program to read x,y coordinates of 3 points and display the area of triangle	
2	Program which generates 100 random numbers in the range of 1 to 100, stored in an array and then displayed using for, while and do..while loops.	
3	Program which determines the largest and the smallest number that can be stored in different data types like short, int, long, float and double. Output when you add 1 to the largest possible integer number that can be stored.	
4	Program which generates 100 random real numbers in the range of 10.0 to 20.0 and sort them in descending order	
5	Program for transporting a square matrix in place	
6	Program to demonstrate string manipulation functions	
7	Program for sorting a list using Bubble sort and then apply Binary search	
8	Program to implement operations on Stack	
9	Program to implement operations on Circular Queue	
10	Program to implement Double Linked list	
11	Program to implement Circular Linked list	
12	Program to sort an array using Quick Sort	
13	Program to sort an array using Merge Sort	
14	Program to implement binary search tree and for implementing in-order, pre-order and post-order traversal using recursion	
15	Program to find DFS of a graph	

// Program1: C program to read x,y coordinates of 3 points and display the area of triangle formed by them.

```
#include<stdio.h>
```

```
#include<math.h>
```

```
#include<conio.h>
```

```
#include<stdlib.h>
```

```
void main()
```

```
{
```

```
float ax,ay,bx,by,cx,cy,col,a,b,c,side,area;
```

```
clrscr();
```

```
printf("\nEnter ax,ay coordinates:");
```

```
scanf("%f%f",&ax,&ay);
```

```
printf("\nEnter bx,by coordinates:");
```

```
scanf("%f%f",&bx,&by);
```

```
printf("\nEnter cx,cy coordinates:");
```

```
scanf("%f%f",&cx,&cy);
```

```
col=ax*(by-cy)+bx*(cy-ay)+cx*(ay-by);
```

```
printf("\nGiven coordinates are (%.2f,%.2f) \t (%.2f,%.2f) \t (%.2f,%.2f)",ax,ay,bx,by,cx,cy);
```

```
if(col==0)
```

```
printf("\nGiven coordinates are collinear. Area of Triangle=0");
```

```
else
```

```
{
```

```
a=sqrt((((ax-bx)*(ax-bx))+((ay-by)*(ay-by))));
```

```
b=sqrt((((bx-cx)*(bx-cx))+((by-cy)*(by-cy))));
```

```
c=sqrt((((cx-ax)*(cx-ax))+((cy-ay)*(cy-ay))));
```

```
side=(a+b+c)/2;
```

```
area=sqrt(side*(side-a)*(side-b)*(side-c));
```

```
printf("\nArea of triangle:%.2f",area);
```

```
}
```

```
getch();
```

```
}
```

// Program 2: C program which generates 100 random numbers in the range of 1 to 100.

Stored in an array and then displayed using for, while and do..while loops.

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
void main()
{
    int c, num[100];
    clrscr();
    printf("Hundred random numbers in [1,100]\n");
    for (c = 0; c < 100; c++)
    {
        num[c]= rand() % 100 + 1;
    }
    printf("\nDisplay numbers using for loop\n");
    for(c=0;c<100;c++)
    printf("%3d",num[c]);

    printf("\nDisplay numbers using while loop\n");
    c=0;
    while(c<100)
    {
        printf("%3d",num[c]);
        c++;
    }
    printf("\nDisplay numbers using do..while loop\n");
    c=0;
    do
    {
        printf("%3d",num[c]);
        c++;
    }while(c<100);
    getch();
}
```

//Program 3: C Program which determines the largest and the smallest number that can be stored

in different data types like short, int, long, float and double. Output when you add 1 to the largest possible integer number that can be stored.

```
#include<limits.h>
```

```
#include<stdio.h>
```

```
#include<float.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int i;
```

```
clrscr();
```

```
printf("\nMin char value:%d Max char value:%d",CHAR_MIN,CHAR_MAX);
```

```
printf("\n\nMin int Value:%d  Max int Value:%d",INT_MIN,INT_MAX);
```

```
i=INT_MAX+1;
```

```
printf("\n\nAfter adding 1 to Max value of Integer, we get %d",i);
```

```
printf("\n\nMin short value:%d  Max short value:%d",SHRT_MIN,SHRT_MAX);
```

```
printf("\n\nMin long value: %ld Max long value:%ld",LONG_MIN,LONG_MAX);
```

```
printf("\n\nMin float value:%e  Max float value:%e",FLT_MIN,FLT_MAX);
```

```
printf("\nMin double value:%le Max double value:%le",DBL_MIN,DBL_MAX);
```

```
printf("\nMin long double value:%Le Max long double  
value:%Le",LDBL_MIN,LDBL_MAX);
```

```
}
```

// Program 4: C program which generates 100 random real numbers in the range of 10.0 to 20.0 and sort them in descending order

```
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>
void insertion_sort(float*,int);
void main()
{
float a[100],l=10.0,u=20.0;
clrscr();
printf("\nBefore sorting\n");
for(i=0;i<100;i++)
{
a[i]= (float(rand())/float((RAND_MAX))*(u-l))+ l;
printf("%.2f \t", a[i]);
}
insertion_sort(a,100);
printf("\nAfter sorting\n");
for (i = 0; i < 100; i++)
{
printf("%.2f \t", a[i]);
}
getch();
}
```

```
void insertion_sort(float *a,int n)
{
float temp;
int i,j;
for(i=1;i<n;i++)
{
temp=a[i];
for(j=i-1;j>=0 && temp>a[j];j--)
a[j+1]=a[j];
```

```
a[j+1]=temp;
```

```
}
```

```
}
```

//Program 5: C function for transporting a square matrix in place

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void transpose(int a[3][3]);
```

```
int i,j;
```

```
void main()
```

```
{
```

```
int mat[3][3];
```

```
clrscr();
```

```
printf("Enter the elements of the matrix\n");
```

```
for(i=0;i<3;i++)
```

```
{
```

```
for(j=0;j<3;j++)
```

```
{
```

```
scanf("%d",&mat[i][j]);
```

```
}
```

```
}
```

```
printf("\nThe matrix\n");
```

```
for(i=0;i<3;i++)
```

```
{
```

```
for(j=0;j<3;j++)
```

```
{
```

```
printf("%d\t",mat[i][j]);
```

```
}
```

```
printf("\n");
```

```
}
```

```
//transpose logic using same matrix
```

```
transpose(mat);
```

```
printf("The transpose of the matrix is\n");
```

```
for(i=0;i<3;i++)
```

```
{
```

```
for(j=0;j<3;j++)
```

```
{
```

```
printf("%d\t",mat[i][j]);
```



```
}  
printf("\n");  
}  
getch();  
}
```

```
void transpose(int a[3][3])  
{  
    int temp;  
    for(i=0;i<3;i++)  
    {  
        for(j=0;j<i;j++)  
        {  
            temp=a[i][j];  
            a[i][j]=a[j][i];  
            a[j][i]=temp;  
        }  
    }  
}
```

// Program 6: C program to demonstrate string manipulation functions

```
#include<string.h>
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
char s1[20],s2[20];
```

```
int l,c;
```

```
clrscr();
```

```
printf("\nEnter a string:");
```

```
gets(s1);
```

```
l=strlen(s1);
```

```
printf("\nLength of string %s is %d",s1,l);
```

```
strcpy(s2,s1); //strcpy(dest,src);
```

```
printf("\nstring s1=%s Copied String s2=%s",s1,s2);
```

```
strcat(s1,s2);
```

```
printf("\nstring after concatenation s1=%s ",s1);
```

```
printf("\n Before Reversing s2=%s",s2);
```

```
strrev(s2);
```

```
printf(" After Reversing s2=%s",s2);
```

```
c=strcmp(s1,s2);
```

```
if(c==0)
```

```
printf("\ns1 and s2 are equal");
```

```
else
```

```
printf("\ns1 and s2 are not equal");
```

```
getch();
```

```
}
```

// Program 7: C program for sorting a list using Bubble sort and then apply Binary search

```
#include<alloc.h>
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>
void bubble_sort(int *,int);
void binary_search(int *,int);
void main()
{
    int *a,n,i;
    clrscr();
    printf("\nHow many elements you want to enter:");
    scanf("%d",&n);
    a=(int*)malloc(n*sizeof(int));
    for(i=0;i<n;i++)
    {
        printf("\nEnterNo.:");
        scanf("%d",&a[i]);
    }
    printf("\nBefore sorting\n");
    for(i=0;i<n;i++)
        printf("%5d",a[i]);
    bubble_sort(a,n);
    printf("\nAfter sorting\n");
    for(i=0;i<n;i++)
        printf("%5d",a[i]);
    binary_search(a,n);
    getch();
}

void bubble_sort(int *a,int n)
{
    int i,j,temp;
    for(i=0;i<n;i++)
        for(j=n-1;j>i;j--)
```

```
if(a[j]<a[j-1])
{
temp=a[j];
a[j]=a[j-1];
a[j-1]=temp;
}
}

void binary_search(int *a,int n)
{
int low,high,mid,m;
printf("\nEnter a element you want to search:");
scanf("%d",&m);
low=0;
high=n-1;

while(low<=high)
{
mid=(low+high)/2;
if(a[mid]==m)
{
printf("\nNumber found at %d position",mid+1);
return;
}
else if(a[mid]<m)
low=mid+1;
else
high=mid-1;
}
printf("Number not found");
}
```

//Program 8: C Program to implement operations on Stack

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<alloc.h>
```

```
#include<stdlib.h>
```

```
int *stack,top=-1,size;
```

```
void push();
```

```
void pop();
```

```
void show();
```

```
void main()
```

```
{
```

```
int ch;
```

```
clrscr();
```

```
printf("\nEnter the sizeof the stack");
```

```
scanf("%d",&size);
```

```
stack=(int*)malloc(size*sizeof(int));
```

```
while(1)
```

```
{
```

```
printf("\nMenu");
```

```
printf("\n1.push");
```

```
printf("\n2.pop");
```

```
printf("\n3.show");
```

```
printf("\n0.Quit");
```

```
printf("\nEnter your choice:");
```

```
scanf("%d",&ch);
```

```
switch(ch)
```

```
{
```

```
case 1:push();
```

```
    break;
```

```
case 2:pop();
```

```
    break;
```

```
case 3:show();
```

```
    break;
```

```
case 0:exit(0);
```

```

default:printf("\nInvalid choice");
}
}
}
void push()
{
int n;
if(top==size-1)
{
printf("Stack is full");
return;
}
printf("\nEnter number to be inserted:");
scanf("%d",&n);
stack[++top]=n;
}
void pop()
{
if(top== -1)
{
printf("Stack is empty");
return;
}
printf("\nPopped number is %d",stack[top--]);
}
void show()
{
int i;
if(top== -1)
{
printf("\nStack is empty");
return;
}
for(i=top;i>=0;i--)

```

```
printf("%5d",stack[i]);  
}
```

//Program 9: C Program to implement operations on Circular Queue

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<alloc.h>
```

```
#include<stdlib.h>
```

```
int *queue,front=-1,rear=-1,size;
```

```
void qinsert();
```

```
void qdelete();
```

```
void qdisplay();
```

```
void main()
```

```
{
```

```
int ch;
```

```
clrscr();
```

```
printf("\nEnter the sizeof the queue");
```

```
scanf("%d",&size);
```

```
queue=(int*)malloc(size*sizeof(int));
```

```
while(1)
```

```
{
```

```
printf("\nQueue Operations");
```

```
printf("\n1.queue insert");
```

```
printf("\n2.queue delete");
```

```
printf("\n3.queue display");
```

```
printf("\n0.Quit");
```

```
printf("\nEnter your choice:");
```

```
scanf("%d",&ch);
```

```
switch(ch)
```

```
{
```

```
case 1:qinsert();
```

```
break;
```

```
case 2:qdelete();
```

```
break;
```

```
case 3:qdisplay();
```

```
break;
```



```

case 0:exit(0);
default:printf("\nInvalid choice");
}
}
}
void qinsert()
{
int n;
if(front==rear+1||(front==0 && rear==size-1))
{
printf("Queue is full");
return;
}
if(rear==size-1)
rear=0;
else if(front==size-1)
front=rear=0;
else
rear=rear+1;
printf("\nEnter number to be inserted:");
scanf("%d",&n);
queue[rear]=n;
}
void qdelete()
{
if(front==size-1)
{
printf("Queue is empty");
return;
}
printf("\nDeleted number is %d",queue[front]);
if(front==rear)
front=rear=-1;
else if(front==size-1)

```

```
front=0;
else
front++;
}
```

```
void qdisplay()
{
int i;
if(front==-1)
{
printf("\nQueue is empty");
return;
}
if(front>rear)
{
for(i=front;i<size;i++)
printf("%5d",queue[i]);
for(i=0;i<=rear;i++)
printf("%5d",queue[i]);
}
else
{
for(i=front;i<=rear;i++)
printf("%5d",queue[i]);
}
}
```

//Program 10: C Program to implement Double Linked list

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<conio.h>
#include<alloc.h>
struct phone
{
char name[20];
long int phno;
struct phone *next,*prev;
}*head=NULL,*tail=NULL;
void insert();
void remove();
void display();
void main()
{
int ch;
clrscr();
while(1)
{
printf("\n1.Insert node");
printf("\n2. delete node");
printf("\n3.View ");
printf("\n0.Quit");
printf("\n\nEnter your choice:");
scanf("%d",&ch);
switch(ch)
{
case 1: insert();
break;
case 2: remove();
break;
case 3: display();
```

```

        break;
case 0: exit(0);
default: printf("\nInvalid choice");
}
}
}
void insert()
{
struct phone *p;
p=(struct phone*)malloc(sizeof(struct phone));
fflush(stdin);
printf("\nEnter Name:");
gets(p->name);
printf("\nEnter Phone Number");
scanf("%ld",&p->phno);
p->next=p->prev=NULL;
if(head==NULL)
head=tail=p;
else
{
tail->next=p;
p->prev=tail;
tail=p;
}
}
void display()
{
struct phone *p;
if(head==NULL)
printf("\nNo Nodes Exists");
else
{
p=head;
printf("\n\nForward Display");

```

```

do
{
printf("\nName=%s Phone Number:%d",p->name,p->phno);
p=p->next;
}while(p!=NULL);
p=tail;
printf("\n\nBackward Display");
do
{
printf("\nName=%s Phone Number:%d",p->name,p->phno);
p=p->prev;
}while(p!=NULL);
}
}
void remove()
{
struct phone *p,*t;
char str[20];
fflush(stdin);
printf("Enter a name you want to delete:");
gets(str);
if(head==NULL)
{
printf("\nNo Nodes Exists");
return;
}
if(strcmp(head->name,str)==0)
{
p=head;
if(head->next==NULL)
head=tail=NULL;
else
{
head=head->next;

```

```
head->prev=NULL;
}
free(p);
printf("\nNode Deleted");
return;
}
p=head;
while(p->next!=tail && strcmp(p->next->name,str)!=0)
p=p->next;
if(strcmp(p->next->name,str)==0)
{
t=p->next;
t->next->prev=t->prev;
t->prev->next=t->next;
if(t==tail)
tail=p;
free(t);
printf("\nNode deleted");
return;
}
printf("\nName not found");
}
```

//Program 11: C Program to implement Circular Linked list

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<conio.h>
#include<alloc.h>
struct phone
{
char name[20];
long int phno;
struct phone *next;
}*head=NULL,*tail=NULL;
void insert();
void remove();
void display();
void main()
{
int ch;
clrscr();
while(1)
{
printf("\n1.Insert node");
printf("\n2. delete node");
printf("\n3.View ");
printf("\n0.Quit");
printf("\n\nEnter your choice:");
scanf("%d",&ch);
switch(ch)
{
case 1: insert();
break;
case 2: remove();
break;
case 3: display();
```

```

        break;
case 0: exit(0);
default: printf("\nInvalid choice");
}
}
}
void insert()
{
struct phone *p;
p=(struct phone*)malloc(sizeof(struct phone));
fflush(stdin);
printf("\nEnter Name:");
gets(p->name);
printf("\nEnter Phone Number");
scanf("%ld",&p->phno);
p->next=NULL;
if(head==NULL)
{
head=tail=p;
p->next=head;
}
else
{
tail->next=p;
tail=p;
tail->next=head;
}
}
void display()
{
struct phone *p;
if(head==NULL)
printf("\nNo Nodes Exists");
else

```



```

{
p=head;
do
{
printf("\nName=%s Phone Number:%d\n",p->name,p->phno);
p=p->next;
}while(p!=head);
}
}

void remove()
{
struct phone *p,*t;
char str[20];
fflush(stdin);
printf("Enter a name you want to delete:");
gets(str);
if(head==NULL)
{
printf("\nNo Nodes Exists");
return;
}
if(strcmp(head->name,str)==0)
{
p=head;
if(head->next==head)
head=tail=NULL;
else
{
head=head->next;
tail->next=head;
}
free(p);
printf("\nNode Deleted");
return;
}

```

```
}  
p=head;  
while(p->next!=tail && strcmp(p->next->name,str)!=0)  
p=p->next;  
if(strcmp(p->next->name,str)==0)  
{  
t=p->next;  
p->next=t->next;  
if(t==tail)  
tail=p;  
free(t);  
printf("\nNode deleted");  
return;  
}  
printf("\nName not found");  
}
```

//Program 12: C Program to sort an array using Quick Sort

```
#include<alloc.h>
#include<stdio.h>
#include<conio.h>
void quick_sort(int*,int);
void qs(int*,int,int);
void main()
{
    int *a,i,n;
    clrscr();
    printf("Enter the number of elements:");
    scanf("%d",&n);
    a=(int*)malloc(n*sizeof(int));
    for(i=0;i<n;i++)
    {
        printf("\n Enter a number:");
        scanf("%d",&a[i]);
    }
    printf("\nBefore Sorting:");
    for(i=0;i<n;i++)
        printf("%5d",a[i]);
    quick_sort(a,n);
    printf("\nAfter Sorting:");
    for(i=0;i<n;i++)
        printf("%5d",a[i]);
    getch();
}

void quick_sort(int *a,int n)
{
    qs(a,0,n-1);
}

void qs(int *a,int left,int right)
{
    int i,j,no,temp;
```

```
i=left;
j=right;
no=a[(left+right)/2];
do
{
while(a[i]<no && i<right)
i++;
while(no<a[j] && j>left)
j--;
if(i<=j)
{
temp=a[i];
a[i]=a[j];
a[j]=temp;
i++;
j--;
}
}while(i<=j);
if(left<j)
qs(a,left,j);
if(right>i)
qs(a,i,right);
}
```

```

//Program 13: C program for Merge Sort
#include<alloc.h>
#include<stdio.h>
#include<conio.h>

int i;

void merge(int a[],int temp[],int lpos,int rpos,int rightend)
{
    int leftend,num,tpos;
    leftend=rpos-1;
    num=(rightend-lpos)+1;
    while(lpos<=leftend && rpos<=rightend)
    {
        if(a[lpos]<=a[rpos])
            temp[tpos++]=a[lpos++];
        else
            temp[tpos++]=a[rpos++];
    }
    while(lpos<=leftend)
        temp[tpos++]=a[lpos++];
    while(rpos<=rightend)
        temp[tpos++]=a[rpos++];
    for(i=0;i<num;i++,rightend--)
        a[rightend]=temp[rightend];
}

void msort(int a[],int temp[],int left,int right)
{
    int center;
    if(left<right)
    {
        center=(left+right)/2;
        msort(a,temp,left,center);
        msort(a,temp,center+1,right);
        merge(a,temp,left,center+1,right);
    }
}

```

```

}
void mergesort(int a[],int n)
{
int *temp;
temp=(int*)malloc(n*sizeof(int));
msort(a,temp,0,n-1);
}
void main()
{
int *a,i,n;
clrscr();
printf("Enter the number of elements:");
scanf("%d",&n);
a=(int*)malloc(n*sizeof(int));
for(i=0;i<n;i++)
{
printf("\n Enter a number:");
scanf("%d",&a[i]);
}
printf("\nBefore Sorting:");
for(i=0;i<n;i++)
printf("%5d",a[i]);
mergesort(a,n);
printf("\nAfter Sorting:");
for(i=0;i<n;i++)
printf("%5d",a[i]);
}

```

//Program 14: Program to implement binary search tree and for implementing in-order, pre-order and post-order traversal using recursion

```
#include<alloc.h>
```

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
struct node
```

```
{
```

```
int info;
```

```
struct node *lc,*rc;
```

```
}*root=NULL,*q[20];
```

```
void preor(struct node *r)
```

```
{
```

```
if(r!=NULL)
```

```
{
```

```
printf("%5d",r->info);
```

```
preor(r->lc);
```

```
preor(r->rc);
```

```
}
```

```
}
```

```
void inor(struct node *r)
```

```
{
```

```
if(r!=NULL)
```

```
{
```

```
inor(r->lc);
```

```
printf("%5d",r->info);
```

```
inor(r->rc);
```

```
}
```

```
}
```

```
void postor(struct node *r)
```

```
{
```

```
if(r!=NULL)
```

```
{
```

```
postor(r->lc);
```

```
postor(r->rc);
```

```

printf("%5d",r->info);
}
}
void tcreate()
{
struct node *temp;
int f=1,r=1,flag=1,data;
printf("\nEnter Data");
scanf("%d",&data);
root=(struct node*)malloc(sizeof(struct node));
root->info=data;
root->lc=NULL;
root->rc=NULL;
q[r]=root;
printf("\nEnter Data");
scanf("%d",&data);
while(data!=1)
{
temp=(struct node*)malloc(sizeof(struct node));
temp->info=data;
temp->lc=temp->rc=NULL;
if(data!=0)
{
if(flag==1)
q[f]->lc=temp;
else
{
q[f]->rc=temp;
++f;
}
++r;
q[r]=temp;
}
else if(flag==0)

```



```

++f;
flag=!(flag);
printf("\nEnter data");
scanf("%d",&data);
}
}

```

```

void main()
{
clrscr();
printf("\ncreating a tree");
tcreate();
printf("\nPreorder of the tree is:");
preor(root);
printf("\nInorder of the tree is:");
inor(root);
printf("\nPostorder of the tree is:");
postor(root);
getch();
}

```

//Program 15: Program to find DFS of a graph

```

#include<alloc.h>
#include<stdio.h>
#include<conio.h>
#define FALSE 0
#define TRUE 1

```

```

typedef struct node
{
int info;
struct node *link;
}node;

```

```

typedef struct table

```

```

{
int visit;
node *nodeptr;
}table;
table *tab[15];
void create(int n)
{
node *nw,*temp;
int i,j,item;
printf("\nEnter the elements of the matrix below:");
for(i=0;i<n;i++)
{
tab[i]=(table*)malloc(sizeof(table));
tab[i]->visit=FALSE;
tab[i]->nodeptr=NULL;
temp=NULL;
for(j=0;j<n;j++)
{
scanf("%d",&item);
if(item)
{
nw=(node*)malloc(sizeof(node));
nw->info=j;
nw->link=NULL;
if(temp)
temp->link=nw;
else
tab[i]->nodeptr=nw;
temp=nw;
}
}
}
}
void dfs_span(int u)

```

```

{
node *cur;
cur=tab[u]->nodeptr;
while(cur)
{
if(tab[cur->info]->visit==FALSE)
{
printf("%d-%d\n",u+1,cur->info+1);
tab[cur->info]->visit=TRUE;
dfs_span(cur->info);
}
cur=cur->link;
}
return;
}

void main()
{
int n,start;
printf("\nEnter the number of nodes");
scanf("%d",&n);
create(n);
printf("\nEnter the starting node:");
scanf("%d",&start);
start--;
tab[start]->visit=TRUE;
printf("\n The DFS spanning tree\n");
dfs_span(start);
getch();
}

```